

Building a New Module: Medical Masters

Based on: CRUD v4 reference implementation ([src/features/react-crud-v4/](#)) **Template doc:** See [implementation-v4.md](#) for the line-by-line walkthrough of the reference. **Date:** 2026-02-16

This guide walks you through creating a **Medical Masters** CRUD module step-by-step, using the v4 architecture. Each step shows exactly what to create, with example code tailored to the Medical Masters domain.

Overview

Medical Masters is a simple lookup table for medical condition types (e.g., Asthma, Diabetes, Allergy). It has:

- A **list** page with search, filter, sort, pagination
- A **create** dialog
- An **edit** dialog
- A **delete** confirmation

Target folder structure

```
src/features/medical-masters/
├── Services/
│   ├── medicalMasters.dtos.ts           Step 1
│   ├── medicalMasters.services.ts       Step 2
│   ├── medicalMasters.queries.ts        Step 3
│   └── medicalMasters.types.ts          Step 4
├── helpers/
│   ├── mapMedicalMasterDtoToRow.ts      Step 5
│   └── buildEditContext.ts              Step 6
├── columns/
│   └── medicalMasterColumns.tsx          Step 7
├── hooks/
│   └── useMedicalMastersPage.ts          Step 8
├── components/
│   ├── MedicalMasterTable.tsx           Step 9
│   ├── MedicalMasterModal.tsx           Step 10
│   ├── MedicalMasterForm.tsx            Step 11
│   ├── DeleteConfirmModal.tsx           Step 12
│   └── StatusPlaceholder.tsx            Step 13
└── MedicalMasters.page.tsx              Step 14
```

Step 1: Define DTOs — [Services/medicalMasters.dtos.ts](#)

What: TypeScript types that mirror the backend API contracts exactly. **Rule:** PascalCase field names to match C# backend. No UI logic.

```
// =====  
// Medical Masters – DTOs (Data Transfer Objects)  
// Shapes that match the API request/response contracts.  
// Keep these 1:1 with the backend – never mix UI concerns here.  
// =====  
  
// — READ (list) —————  
  
export type GetMedicalMastersResponse = {  
  MedicalConditions: MedicalMasterDto[];  
};  
  
export type MedicalMasterDto = {  
  MedicalConditionId: number;  
  ConditionName: string;  
  ConditionCode: string;  
  Category: string; // e.g. "Chronic", "Allergy", "Disability"  
  Description: string | null;  
  IsActive: boolean;  
  SortOrder: number;  
  CreatedDate: string; // ISO date string from backend  
  ModifiedDate: string | null;  
};  
  
// — READ (categories for dropdown) —————  
  
export type GetCategoriesResponse = {  
  Categories: CategoryDto[];  
};  
  
export type CategoryDto = {  
  CategoryId: number;  
  CategoryName: string;  
};  
  
// — CREATE —————  
  
export type CreateMedicalMasterRequest = {  
  conditionName: string;  
  conditionCode: string;  
  categoryId: number;  
  description: string;  
  isActive: boolean;  
  sortOrder: number;  
};  
  
export type CreateMedicalMasterResponse = {  
  MedicalConditionId: number;  
};  
  
// — UPDATE —————  
  
export type UpdateMedicalMasterRequest = {
```

```

    medicalConditionId: number;
    conditionName: string;
    conditionCode: string;
    categoryId: number;
    description: string;
    isActive: boolean;
    sortOrder: number;
  };

  export type UpdateMedicalMasterResponse = {
    Success: boolean;
  };

  // — DELETE —————

  export type DeleteMedicalMasterRequest = {
    medicalConditionId: number;
  };

```

What to check

- ☐ Every field matches the C# model exactly (PascalCase for responses, camelCase for request bodies if that's your backend convention)
- ☐ Nullable fields use `string | null` or `number | null`
- ☐ No UI types mixed in (no `ItemRow`, no `FormValues`)

Step 2: Define Services — `Services/medicalMasters.services.ts`

What: One function per API endpoint. Pure HTTP calls. **Rule:** No React, no state, no business logic. Just `apiClient.get/post`.

```

// =====
// Medical Masters – API Service Layer
// Each function maps to exactly one API endpoint.
// No business logic here – just HTTP calls via the shared apiClient.
// =====

import apiClient from "@/features/common/API/apiClient";
import type {
  GetMedicalMastersResponse,
  GetCategoriesResponse,
  CreateMedicalMasterRequest,
  CreateMedicalMasterResponse,
  UpdateMedicalMasterRequest,
  UpdateMedicalMasterResponse,
  DeleteMedicalMasterRequest,
} from "../medicalMasters.dtos";

// — Server filter params —————

```

```
export type GetMedicalMastersParams = {
  category?: string;
  isActive?: boolean;
};

// — READ —————

export const getMedicalMasters = async (
  params?: GetMedicalMastersParams,
): Promise<GetMedicalMastersResponse> => {
  const response = await apiClient.get<GetMedicalMastersResponse>(
    "/api/Medical/GetMedicalConditions",
    { params },
  );
  return response.data;
};

export const getCategories = async (): Promise<GetCategoriesResponse> => {
  const response = await apiClient.get<GetCategoriesResponse>(
    "/api/Medical/GetMedicalCategories",
  );
  return response.data;
};

// — CREATE —————

export const createMedicalMaster = async (
  payload: CreateMedicalMasterRequest,
): Promise<CreateMedicalMasterResponse> => {
  const response = await apiClient.post<CreateMedicalMasterResponse>(
    "/api/Medical/CreateMedicalCondition",
    payload,
  );
  return response.data;
};

// — UPDATE —————

export const updateMedicalMaster = async (
  payload: UpdateMedicalMasterRequest,
): Promise<UpdateMedicalMasterResponse> => {
  const response = await apiClient.post<UpdateMedicalMasterResponse>(
    "/api/Medical/UpdateMedicalCondition",
    payload,
  );
  return response.data;
};

// — DELETE —————

export const deleteMedicalMaster = async (
  payload: DeleteMedicalMasterRequest,
): Promise<void> => {
```

```
await apiClient.post("/api/Medical/DeleteMedicalCondition", payload);
};
```

What to check

- ☐ Endpoints match your backend controller routes
- ☐ Each function is `async` and returns `response.data`
- ☐ `GetMedicalMastersParams` matches the query string params your API accepts
- ☐ All DTO imports come from `./medicalMasters.dtos`

Step 3: Define Query Hooks — `Services/medicalMasters.queries.ts`

What: React Query hooks — one per API operation. **Rule:** Mutations invalidate the list query on success. Show toasts for user feedback.

```
// =====
// Medical Masters – React Query Hooks
// One hook per API operation. Mutations invalidate the list query on success.
// =====

import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import type { AxiosError } from "axios";
import { toast } from "react-toastify";

import {
  getMedicalMasters,
  getCategories,
  createMedicalMaster,
  updateMedicalMaster,
  deleteMedicalMaster,
  type GetMedicalMastersParams,
} from "../medicalMasters.services";
import type {
  GetMedicalMastersResponse,
  GetCategoriesResponse,
  CreateMedicalMasterRequest,
  CreateMedicalMasterResponse,
  UpdateMedicalMasterRequest,
  UpdateMedicalMasterResponse,
  DeleteMedicalMasterRequest,
} from "../medicalMasters.dtos";

// — Query Keys (single source of truth) —————

const QUERY_KEYS = {
  items: ["getMedicalConditions"] as const,
  categories: ["getMedicalCategories"] as const,
};
```

```
// — READ —————  
  
export const useGetMedicalMasters = (params?: GetMedicalMastersParams) =>  
  useQuery<GetMedicalMastersResponse, AxiosError>({  
    queryKey: [...QUERY_KEYS.items, params],  
    queryFn: () => getMedicalMasters(params),  
    enabled: !!params, // won't fire until user clicks Search  
  });  
  
export const useGetCategories = () =>  
  useQuery<GetCategoriesResponse, AxiosError>({  
    queryKey: [...QUERY_KEYS.categories],  
    queryFn: getCategories,  
    staleTime: 5 * 60 * 1000, // categories rarely change  
  });  
  
// — CREATE —————  
  
export const useCreateMedicalMaster = () => {  
  const queryClient = useQueryClient();  
  
  return useMutation<  
    CreateMedicalMasterResponse,  
    AxiosError,  
    CreateMedicalMasterRequest  
  >({  
    mutationFn: createMedicalMaster,  
    onSuccess: async () => {  
      toast.success("Medical condition created.");  
      await queryClient.invalidateQueries({ queryKey: QUERY_KEYS.items });  
    },  
    onError: () => {  
      toast.error("Unable to create medical condition. Please try again.");  
    },  
  });  
};  
  
// — UPDATE —————  
  
export const useUpdateMedicalMaster = () => {  
  const queryClient = useQueryClient();  
  
  return useMutation<  
    UpdateMedicalMasterResponse,  
    AxiosError,  
    UpdateMedicalMasterRequest  
  >({  
    mutationFn: updateMedicalMaster,  
    onSuccess: async () => {  
      toast.success("Medical condition updated.");  
      await queryClient.invalidateQueries({ queryKey: QUERY_KEYS.items });  
    },  
    onError: () => {
```

```

        toast.error("Unable to update medical condition. Please try again.");
    },
  });
};

// — DELETE —————

export const useDeleteMedicalMaster = (options?: {
  onDeleted?: () => void;
}) => {
  const queryClient = useQueryClient();

  return useMutation<void, AxiosError, DeleteMedicalMasterRequest>({
    mutationFn: deleteMedicalMaster,
    onSuccess: async () => {
      toast.success("Medical condition deleted.");
      await queryClient.invalidateQueries({ queryKey: QUERY_KEYS.items });
      options?.onDeleted?.();
    },
    onError: () => {
      toast.error("Unable to delete medical condition. Please try again.");
    },
  });
};

```

Key patterns

- `enabled: !!params` — "Search button" pattern. Query doesn't fire until user clicks Search.
- `QUERY_KEYS` — single source of truth. Never hardcode query key strings elsewhere.
- All mutations invalidate `QUERY_KEYS.items` — table auto-refreshes after any CUD operation.
- `useDeleteMedicalMaster` accepts `onDeleted` callback — lets the delete modal close itself after success.

Step 4: Define UI Types — `Services/medicalMasters.types.ts`

What: UI-facing domain types. Separate from DTOs. **Rule:** These types are for components — not for the API.

```

// =====
// Medical Masters – Domain Types
// UI-facing types used by components. Separate from DTOs.
// =====

/** Shape used to pre-populate the modal for create or edit. */
export type MedicalMasterEditContext = {
  mode: "create" | "update";
  medicalConditionId?: number;
  conditionName: string;
  conditionCode: string;
}

```

```

    categoryId: number;
    description: string;
    isActive: boolean;
    sortOrder: number;
  };

  /** Form values managed by react-hook-form inside the modal. */
  export type MedicalMasterFormValues = {
    conditionName: string;
    conditionCode: string;
    categoryId: string | null; // string because <Select> values are strings
    description: string;
    isActive: boolean;
    sortOrder: number;
  };

```

Why separate from DTOs?

- `categoryId` is `number` in the DTO but `string | null` in form values — because Radix `<Select>` works with strings.
- `mode: "create" | "update"` discriminant exists only in the UI — the API doesn't know about it.
- If the API changes field names, only the DTOs and mapper change — form values stay the same.

Step 5: Define Row Mapper — `helpers/mapMedicalMasterDtoToRow.ts`

What: Pure function that transforms API DTO → table row. Co-locates the row type with its mapper.

Rule: No React, no hooks, no side effects. Easy to unit test.

```

// =====
// Pure mapper: API DTO → UI row
// No side effects. Easy to unit test.
// =====

import dayjs from "dayjs";
import type { MedicalMasterDto } from "../Services/medicalMasters.dtos";

/** A single row displayed in the table. */
export type MedicalMasterRow = {
  id: number; // MedicalConditionId
  conditionName: string;
  conditionCode: string;
  category: string;
  description: string;
  isActive: boolean;
  sortOrder: number;
  createdAt: string;
  modifiedDate: string | null;
};

```



```
export const mapMedicalMasterDtoToRow = (
  dto: MedicalMasterDto,
): MedicalMasterRow => ({
  id: dto.MedicalConditionId,
  conditionName: dto.ConditionName,
  conditionCode: dto.ConditionCode,
  category: dto.Category,
  description: dto.Description ?? "",
  isActive: dto.IsActive,
  sortOrder: dto.SortOrder,
  createdAt: dayjs(dto.CreatedDate).format("DD/MM/YYYY"),
  modifiedDate: dto.ModifiedDate
    ? dayjs(dto.ModifiedDate).format("DD/MM/YYYY")
    : null,
});
```

What to check

- ☐ Every field the table needs is in `MedicalMasterRow`
- ☐ PascalCase (DTO) → camelCase (Row) for every field
- ☐ Dates formatted here, not in column defs
- ☐ Nullable fields handled with `?? ""` or null guards

Step 6: Define Edit Context Builder — `helpers/buildEditContext.ts`

What: Pure function that builds the edit modal's default values from a DTO. **Rule:** No React. Testable with `expect(buildEditContext(mockDto)).toEqual(...)`.

```
// =====
// Pure helper: builds a MedicalMasterEditContext from a DTO for the edit modal.
// No React, no side effects. Easy to unit test.
// =====

import type { MedicalMasterDto } from "../Services/medicalMasters.dtos";
import type { MedicalMasterEditContext } from "../Services/medicalMasters.types";

export const buildEditContext = (
  dto: MedicalMasterDto,
): MedicalMasterEditContext => ({
  mode: "update",
  medicalConditionId: dto.MedicalConditionId,
  conditionName: dto.ConditionName,
  conditionCode: dto.ConditionCode,
  categoryId: 0, // list API may not return categoryId – fetch from
  detail endpoint
  description: dto.Description ?? "",
  isActive: dto.IsActive,
  sortOrder: dto.SortOrder,
});
```

Note

If your list API returns `CategoryId`, map it directly instead of defaulting to `0`. The v4 reference has the same pattern — the list endpoint doesn't always return every field needed for edit.

Step 7: Define Columns — `columns/medicalMasterColumns.tsx`

What: All column definitions, default visibility, filterable columns, and row actions. **Rule:** To add/remove/reorder columns, edit this file only.

```
// =====
// Medical Masters – Column Definitions
// To add/remove/reorder columns, edit this file only.
// =====

import type { ColumnDef, VisibilityState } from "@tanstack/react-table";
import { EditIcon, MoreHorizontalIcon, Trash2Icon } from "lucide-react";

import { Button } from "@components/ui/v1/button";
import { Checkbox } from "@components/ui/v1/checkbox";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from "@components/ui/v1/dropdown-menu";
import { TableHeaderButton } from "@components/layouts/v1/Table.layout";
import type { FilterableColumn } from
"@components/layouts/v1/TableFilterBuilder";

import type { MedicalMasterRow } from "../helpers/mapMedicalMasterDtoToRow";

// — Filterable columns for TableFilterBuilder —————

export const FILTERABLE_COLUMNS: FilterableColumn[] = [
  { id: "conditionName", label: "Condition Name", type: "string" },
  { id: "conditionCode", label: "Code", type: "string" },
  { id: "category", label: "Category", type: "string" },
];

// — Default column visibility —————

export const DEFAULT_COLUMN_VISIBILITY: VisibilityState = {
  sortOrder: false,
  createDate: false,
  modifiedDate: false,
};
```

```
// — Row actions —————

export type ActionItem<T> = {
  label: string;
  icon?: React.ComponentType<{ className?: string }>;
  onClick: (row: T) => void;
  variant?: "destructive";
};

export type ActionsConfig<T> = {
  quick: ActionItem<T>[];
  menu: (ActionItem<T> | "separator")[];
};

// — Column factory —————

type ColumnConfig = {
  actions?: ActionsConfig<MedicalMasterRow>;
};

export const buildMedicalMasterColumns = (
  config: ColumnConfig = {},
): ColumnDef<MedicalMasterRow>[] => {
  const { actions } = config;

  return [
    // — Select checkbox —————
    {
      id: "select",
      size: 1,
      enableSorting: false,
      enableGlobalFilter: false,
      header: ({ table }) => (
        <Checkbox
          checked={table.getIsAllPageRowsSelected()}
          onChange={(v) => table.toggleAllPageRowsSelected(!v)}
          aria-label="Select all rows"
        />
      ),
      cell: ({ row }) => (
        <Checkbox
          checked={row.getIsSelected()}
          onChange={(v) => row.toggleSelected(!v)}
          aria-label={`Select ${row.original.conditionName}`}
        />
      ),
    },
  ],
};

// — Condition Name —————
{
  accessorKey: "conditionName",
  size: 20,
  header: ({ column }) => (
    <TableHeaderButton column={column}>Condition Name</TableHeaderButton>
  )
}
```

```
    ),
  },

  // — Code —————
  {
    accessorKey: "conditionCode",
    size: 10,
    header: ({ column }) => (
      <TableHeaderButton column={column}>Code</TableHeaderButton>
    ),
    cell: ({ getValue }) => (
      <span className="text-muted-foreground">{getValue<string>()}</span>
    ),
  },

  // — Category —————
  {
    accessorKey: "category",
    size: 12,
    header: ({ column }) => (
      <TableHeaderButton column={column}>Category</TableHeaderButton>
    ),
  },

  // — Description —————
  {
    accessorKey: "description",
    size: 20,
    enableSorting: false,
    header: "Description",
    cell: ({ getValue }) => (
      <span className="text-muted-foreground line-clamp-1">
        {getValue<string>()}
      </span>
    ),
  },

  // — Active —————
  {
    accessorKey: "isActive",
    size: 6,
    header: "Active",
    cell: ({ getValue }) => (
      <span>{getValue<boolean>()} ? "Yes" : "No"</span>
    ),
  },

  // — Sort Order —————
  {
    accessorKey: "sortOrder",
    size: 6,
    header: "Sort Order",
  },
```

```

// — Created Date —————
{
  accessorKey: "createdDate",
  size: 8,
  header: "Created",
  cell: ({ getValue }) => (
    <span className="text-muted-foreground">{getValue<string>()}</span>
  ),
},

// — Modified Date —————
{
  accessorKey: "modifiedDate",
  size: 8,
  header: "Modified",
  cell: ({ getValue }) => (
    <span className="text-muted-foreground">
      {getValue<string | null>() ?? "-"}
    </span>
  ),
},

// — Actions (conditional) —————
...(actions
  ? [
    {
      id: "actions",
      size: 1 + actions.quick.length,
      enableSorting: false,
      enableGlobalFilter: false,
      header: "",
      cell: ({ row }: { row: { original: MedicalMasterRow } }) => (
        <div className="flex items-center justify-end gap-1">
          {actions.quick.map((action) => (
            <Button
              key={action.label}
              variant="ghost"
              size="icon-sm"
              onClick={() => action.onClick(row.original)}
              aria-label={action.label}
            >
              {action.icon && <action.icon className="h-4 w-4" />}
            </Button>
          ))}
          {actions.menu.length > 0 && (
            <DropdownMenu>
              <DropdownMenuTrigger asChild>
                <Button variant="ghost" size="icon-sm" aria-label="More
actions">
                  <MoreHorizontalIcon className="h-4 w-4" />
                </Button>
              </DropdownMenuTrigger>
              <DropdownMenuContent align="end">
                {actions.menu.map((item, i) =>

```

```

        item === "separator" ? (
          <DropdownMenuSeparator key={`sep-${i}`} />
        ) : (
          <DropdownMenuItem
            key={item.label}
            onClick={() => item.onClick(row.original)}
            variant={item.variant}
          >
            {item.icon && <item.icon className="mr-2 h-4 w-4" />}
            {item.label}
          </DropdownMenuItem>
        ),
      )}
    </DropdownMenuContent>
  </DropdownMenu>
)}
</div>
),
} satisfies ColumnDef<MedicalMasterRow>,
]
: []),
];
};

```

What to check

- ☐ `FILTERABLE_COLUMNS` ids match `accessorKey` values
- ☐ `DEFAULT_COLUMN_VISIBILITY` hides columns that are less important
- ☐ `size` values are proportions (not pixels) — they'll be normalized to percentages
- ☐ Every interactive element has `aria-label`

Step 8: Define Page Hook — `hooks/useMedicalMastersPage.ts`

What: All state, handlers, and derived data. The page component only calls this hook. **Rule:** No JSX. No styling. Just state management.

```

// =====
// Medical Masters – Page Hook
// All state, handlers, and derived data for the page live here.
// The page component only calls this hook and renders JSX.
// =====

import { useCallback, useMemo, useReducer, useState } from "react";

import {
  useGetMedicalMasters,
  useGetCategories,
} from "../Services/medicalMasters.queries";
import type { GetMedicalMastersParams } from
"../Services/medicalMasters.services";

```

```

import { mapMedicalMasterDtoToRow } from "../helpers/mapMedicalMasterDtoToRow";
import { buildEditContext } from "../helpers/buildEditContext";
import type { MedicalMasterRow } from "../helpers/mapMedicalMasterDtoToRow";
import type { MedicalMasterEditContext } from "../Services/medicalMasters.types";
import type { MedicalMasterDto } from "../Services/medicalMasters.dtos";

// — Reducer —

type ModalState = {
  modalOpen: boolean;
  editCtx: MedicalMasterEditContext | null;
  deleteConfirmOpen: boolean;
};

type ModalAction =
  | { type: "OPEN_CREATE" }
  | { type: "OPEN_EDIT"; ctx: MedicalMasterEditContext }
  | { type: "CLOSE_MODAL" }
  | { type: "OPEN_DELETE_CONFIRM" }
  | { type: "CLOSE_DELETE_CONFIRM" };

const initialModalState: ModalState = {
  modalOpen: false,
  editCtx: null,
  deleteConfirmOpen: false,
};

const modalReducer = (state: ModalState, action: ModalAction): ModalState => {
  switch (action.type) {
    case "OPEN_CREATE":
      return {
        ...state,
        modalOpen: true,
        editCtx: {
          mode: "create",
          conditionName: "",
          conditionCode: "",
          categoryId: 0,
          description: "",
          isActive: true, // default new conditions to active
          sortOrder: 0,
        },
      };
    case "OPEN_EDIT":
      return { ...state, modalOpen: true, editCtx: action.ctx };
    case "CLOSE_MODAL":
      return { ...state, modalOpen: false, editCtx: null };
    case "OPEN_DELETE_CONFIRM":
      return { ...state, deleteConfirmOpen: true };
    case "CLOSE_DELETE_CONFIRM":

```

```
        return { ...state, deleteConfirmOpen: false };

        default:
            return state;
    }
};

// — Hook —————

export const useMedicalMastersPage = () => {
    const [modal, dispatch] = useReducer(modalReducer, initialModalState);

    // — Server filter state —————
    const [serverParams, setServerParams] = useState<
        GetMedicalMastersParams | undefined
    >(undefined);

    // — Data fetching —————
    const {
        data: itemsResponse,
        isLoading,
        isFetching,
        isError,
    } = useGetMedicalMasters(serverParams);
    const { data: categoriesResponse } = useGetCategories();

    const items = useMemo<MedicalMasterDto[]>(
        () => itemsResponse?.MedicalConditions ?? [],
        [itemsResponse],
    );
    const rows = useMemo<MedicalMasterRow[]>(
        () => items.map(mapMedicalMasterDtoToRow),
        [items],
    );

    const categories = useMemo(
        () => categoriesResponse?.Categories ?? [],
        [categoriesResponse],
    );

    // — Handlers —————

    const handleServerSearch = useCallback(
        (params: GetMedicalMastersParams) => {
            setServerParams(params);
        },
        [],
    );

    const handleCreate = useCallback(() => {
        dispatch({ type: "OPEN_CREATE" });
    }, []);

    const handleEdit = useCallback(
```



```

    (itemId: number) => {
      const dto = items.find((i) => i.MedicalConditionId === itemId);
      if (!dto) return;
      dispatch({ type: "OPEN_EDIT", ctx: buildEditContext(dto) });
    },
    [items],
  );

  const handleCloseModal = useCallback(() => {
    dispatch({ type: "CLOSE_MODAL" });
  }, []);

  const handleOpenDeleteConfirm = useCallback(() => {
    dispatch({ type: "OPEN_DELETE_CONFIRM" });
  }, []);

  const handleCloseDeleteConfirm = useCallback(() => {
    dispatch({ type: "CLOSE_DELETE_CONFIRM" });
  }, []);

  // — Return —————

  return {
    // Data
    rows,
    categories,
    isLoading,
    isFetching,
    isError,
    serverParams,
    hasSearched: !!serverParams,

    // Modal state
    modalOpen: modal.modalOpen,
    editCtx: modal.editCtx,
    deleteConfirmOpen: modal.deleteConfirmOpen,

    // Handlers
    handleServerSearch,
    handleCreate,
    handleEdit,
    handleCloseModal,
    handleOpenDeleteConfirm,
    handleCloseDeleteConfirm,
  };
};

```

What changed from v4 reference

- `OPEN_CREATE` defaults: `conditionName: ""`, `isActive: true`, etc. — match your domain defaults
- `handleEdit` finds by `MedicalConditionId` instead of `StudentNumber`
- `items` extracts from `MedicalConditions` instead of `StudentSummary`

- Everything else is the same pattern
-

Step 9-13: Components

These follow the exact same patterns as the v4 reference. The key changes are:

Step 9: `MedicalMasterTable.tsx`

- Copy `ItemTable.tsx` from v4
- Replace `ItemRow` → `MedicalMasterRow`
- Replace `buildItemColumns` → `buildMedicalMasterColumns`
- Replace `GetItemsParams` → `GetMedicalMastersParams`
- Update server filter dropdown: Category instead of Year
- Update `yearOptions` prop → `categoryOptions` prop
- Update skeleton column count to match your visible columns

Step 10: `MedicalMasterModal.tsx`

- Copy `ItemModal.tsx` from v4
- Replace `ItemEditContext` → `MedicalMasterEditContext`
- Replace `ItemFormValues` → `MedicalMasterFormValues`
- Replace `useCreateItem` → `useCreateMedicalMaster`
- Replace `useUpdateItem` → `useUpdateMedicalMaster`
- Update `buildDefaults()`: map `categoryId` number → string for `<Select>`
- Update `buildPayload()`: map `categoryId` string → number for API
- Update dialog title: "Create Medical Condition" / "Edit Medical Condition"

Step 11: `MedicalMasterForm.tsx`

- Copy `ItemForm.tsx` from v4
- Replace fields with: Condition Name (Input), Code (Input), Category (Select), Description (Textarea), Active (Checkbox), Sort Order (Input type="number")
- Remove date picker and teacher search — not needed for this module
- Update validation rules to match your requirements

Step 12: `DeleteConfirmModal.tsx`

- Copy directly from v4 — **almost no changes needed**
- Replace `useDeleteItem` → `useDeleteMedicalMaster`
- Update `DeleteItemRequest` → `DeleteMedicalMasterRequest`
- Update the ID field: `{ medicalConditionId: id }` instead of `{ studentActivityId: id }`
- Update labels: "Medical Condition" instead of "Item"

Step 13: `StatusPlaceholder.tsx`

- **Copy directly from v4 — no changes needed.** This component is fully generic.
-

Step 14: Define Page Orchestrator — `MedicalMasters.page.tsx`

What: The orchestrator. Calls the hook, renders child components. **Rule:** No business logic. No state management beyond the delete bridge.

```
// =====
// Medical Masters – Page Component
// This is the orchestrator. It calls the hook and renders child components.
// No business logic, no state management – just layout and wiring.
// =====

import { useCallback, useState } from "react";

import {
  PageContent,
  PageTitle,
  SingleColumnPage,
} from "@components/layouts/v1/SingleColumnPage.layout";

import { useMedicalMastersPage } from "../hooks/useMedicalMastersPage";
import { StatusPlaceholder } from "../components/StatusPlaceholder";
import { MedicalMasterTable } from "../components/MedicalMasterTable";
import { MedicalMasterModal } from "../components/MedicalMasterModal";
import { DeleteConfirmModal } from "../components/DeleteConfirmModal";

const MedicalMastersPage = () => {
  const {
    rows,
    categories,
    isLoading,
    isFetching,
    isError,
    hasSearched,
    modalOpen,
    editCtx,
    deleteConfirmOpen,
    handleServerSearch,
    handleCreate,
    handleEdit,
    handleCloseModal,
    handleOpenDeleteConfirm,
    handleCloseDeleteConfirm,
  } = useMedicalMastersPage();

  // Track which IDs are pending deletion
  const [pendingDeleteIds, setPendingDeleteIds] = useState<number[]>([]);

  const handleDeleteSelected = useCallback(
    (ids: number[]) => {
      setPendingDeleteIds(ids);
      handleOpenDeleteConfirm();
    },
    [handleOpenDeleteConfirm],
  );
};
```

```

const handleDeleteCompleted = useCallback(() => {
  setPendingDeleteIds([]);
}, []);

return (
  <SingleColumnPage windowTitle="Medical Masters">
    <PageTitle>Medical Masters</PageTitle>

    <PageContent>
      {/* Error state */}
      {isError && (
        <StatusPlaceholder variant="error">
          Unable to load medical conditions. Please try again later.
        </StatusPlaceholder>
      )}

      {/* Table */}
      {!isError && (
        <MedicalMasterTable
          rows={rows}
          isLoading={isLoading}
          isFetching={isFetching}
          hasSearched={hasSearched}
          categoryOptions={categories.map((c) => c.CategoryName)}
          onServerSearch={handleServerSearch}
          onCreate={handleCreate}
          onEdit={handleEdit}
          onDeleteSelected={handleDeleteSelected}
        />
      )}

      {/* Create / Edit Modal */}
      {editCtx && (
        <MedicalMasterModal
          open={modalOpen}
          onClose={handleCloseModal}
          editCtx={editCtx}
          categories={categories}
        />
      )}

      {/* Delete Confirmation Modal */}
      <DeleteConfirmModal
        open={deleteConfirmOpen}
        onClose={handleCloseDeleteConfirm}
        selectedIds={pendingDeleteIds}
        onDelete={handleDeleteCompleted}
      />
    </PageContent>
  </SingleColumnPage>
);
};

```

```
export default MedicalMastersPage;
```

Step 15: Register the Route

Add the page to your router configuration:

```
// In your router config file
const MedicalMastersPage = lazy(
  () => import("@/features/medical-masters/MedicalMasters.page"),
);

// Add to routes array
{ path: "/medical-masters", element: <MedicalMastersPage /> }
```

Checklist — Before You Ship

#	Check	Status
1	DTOs match backend API contracts exactly	[]
2	Service endpoints are correct	[]
3	Query keys are unique (no collision with other features)	[]
4	<code>enabled: !!params</code> prevents auto-fetch on mount	[]
5	Mapper handles all nullable fields	[]
6	Column <code>aria-label</code> on checkboxes and action buttons	[]
7	Form fields have <code>Label</code> with <code>htmlFor</code>	[]
8	Delete modal uses <code>AlertDialog</code> (not <code>Dialog</code>)	[]
9	Toast messages are user-friendly	[]
10	No <code>dangerouslySetInnerHTML</code> anywhere	[]
11	No hardcoded dropdown options — fetch from server	[]
12	Route registered and lazy-loaded	[]
13	Page title set via <code>SingleColumnPage windowTitle</code>	[]
14	Build passes with no TypeScript errors	[]

Quick Reference — File Change Map

When something changes, you only need to touch specific files:

Change	Files to edit
API contract changes	medicalMasters.dtos.ts only
New endpoint	medicalMasters.services.ts + medicalMasters.queries.ts
New table column	medicalMasterColumns.tsx + mapMedicalMasterDtoToRow.ts
New form field	MedicalMasterForm.tsx + medicalMasters.types.ts
New server filter	medicalMasters.services.ts (params) + MedicalMasterTable.tsx (dropdown)
Change page state	useMedicalMastersPage.ts only
Change button style	components/ui/v1/button.tsx (affects ALL buttons globally)
Change table layout	components/layouts/v1/Table.layout.tsx (affects ALL tables globally)